SQL JOIN Practice Tasks

For Beginners - Oracle HR Schema - A1 English

Tables You Will Use

In these tasks, you will work with the Oracle HR database. This database has many tables. Here are the most important tables and their columns. You will use these tables in all tasks below.

Table	Key Columns	What It Stores
`employees`	employee_id, first_name, last_name, email, salary, department_id, manager_id, job_id, hire_date	All workers in the company
`departments`	department_id, department_name, manager_id, location_id	Company departments (IT, Sales, HR, etc.)
`jobs`	job_id, job_title, min_salary, max_salary	Job roles and salary ranges
`locations`	location_id, street_address, city, country_id	Office addresses
`countries`	country_id, country_name, region_id	Country names
`regions`	region_id, region_name	World regions (Americas, Europe, etc.)
`job_history`	employee_id, start_date, end_date, job_id, department_id	Past jobs of employees

Part 1: INNER JOIN Tasks

INNER JOIN returns only rows that have a match in BOTH tables. If a row in the left table has no match in the right table, that row will NOT appear in the result. This is the most common type of JOIN in SQL.

Task 1: Basic Two-Table JOIN [10 pts]

Write a query to show the **first name**, **last name**, and **department name** for all employees. Use the **employees** table and the **departments** table. Join them on **department_id**.

> **Hint:** Use: SELECT e.first_name, e.last_name, d.department_name FROM employees e JOIN departments d ON e.department_id = d.department_id

Your Answer:

Task 2: JOIN with WHERE Filter [15 pts]

Write a query to show the **employee last name**, **job title**, and **salary**. Join **employees** with **jobs**. Show only employees who earn **more than 10000**.

> **Hint:** Use: SELECT e.last_name, j.job_title, e.salary FROM employees e JOIN jobs j ON e.job_id = j.job_id WHERE e.salary > 10000

Your Answer:

Task 3: Three-Table JOIN [15 pts]

Write a query to show the **employee last name**, **department name**, and **city** where the department is located. You need to join **employees**, **departments**, and **locations**.

> **Hint:** Join employees to departments on department_id, then join departments to locations on location_id.

Your Answer:

Task 4: JOIN with ORDER BY [10 pts]

Write a query to show the **employee last name** and **department name**. Order the result by **salary** from highest to lowest.

> **Hint:** Join employees and departments. Use ORDER BY e.salary DESC at the end.

Your Answer:

Part 2: LEFT (OUTER) JOIN Tasks

LEFT JOIN returns ALL rows from the left table, even if there is no match in the right table. When there is no match, the result shows NULL values for the right table columns. This is very useful when you want to see all records from one table, with matching data from another table when it exists.

Task 5: Find Employees Without a Department [10 pts]

Some employees do not have a department_id. Their department_id is NULL. Write a query to show the **last name** and **department name** of ALL employees, including those without a department. Use LEFT JOIN.

> **Hint:** Use LEFT JOIN and you will see NULL for department_name when the employee has no department.

Your Answer:

Task 6: Count Employees Per Department (Include Zero) [15 pts]

Write a query to show each **department name** and the **number of employees** in that department. Use LEFT JOIN so that departments with zero employees also appear in the result. Use COUNT(e.employee_id) and GROUP BY.

> **Hint:** Use LEFT JOIN from departments to employees. COUNT(e.employee_id) counts only non-NULL values.

Your Answer:

Task 7: Departments with No Employees [15 pts]

Write a query to find departments that have **no employees** at all. Show the **department name** only.

> **Hint:** Use LEFT JOIN and then add WHERE e.employee_id IS NULL to find departments with no match.

Your Answer:

Part 3: RIGHT (OUTER) JOIN Tasks

RIGHT JOIN is the opposite of LEFT JOIN. It returns ALL rows from the right table, even if there is no match in the left table. When there is no match, the left table columns show NULL. Note: You can always swap the table order and use LEFT JOIN instead of RIGHT JOIN.

Task 8: All Departments with Employee Names [10 pts]

Write a query using RIGHT JOIN to show **department name** and **employee last name**. All departments must appear, even those with no employees. Put **employees** on the LEFT and **departments** on the RIGHT in your FROM clause.

> **Hint:** FROM employees e RIGHT JOIN departments d ON e.department_id = d.department_id

Your Answer:

Task 9: Convert RIGHT JOIN to LEFT JOIN [10 pts]

Rewrite the query from Task 8 using only LEFT JOIN (no RIGHT JOIN). The result must be the same. Show the **department name** and **employee last name**.

> **Hint:** Swap the tables: FROM departments d LEFT JOIN employees e ON d.department_id = e.department_id

Your Answer:

Part 4: FULL OUTER JOIN Tasks

FULL OUTER JOIN returns ALL rows from BOTH tables. When there is a match, the rows are combined. When there is no match, the missing side shows NULL. This is useful when you want to see everything from both tables, with matches where they exist.

Task 10: Employees and Departments - Complete List [10 pts]

Write a query to show **employee last name** and **department name**. Use FULL OUTER JOIN so that ALL employees and ALL departments appear in the result. If an employee has no department, department_name will be NULL. If a department has no employees, last_name will be NULL.

> **Hint:** FULL OUTER JOIN returns everything from both sides. Use FROM employees e FULL OUTER JOIN departments d ON ...

Your Answer:

Part 5: CROSS JOIN Tasks

CROSS JOIN combines every row from the first table with every row from the second table. There is no ON condition. If table A has 3 rows and table B has 4 rows, the result has 3 x 4 = 12 rows. This is also called a "Cartesian product".

Task 11: List All Possible Employee-Department Pairs [10 pts]

Write a query to create a list of every **employee last name** with every **department name**. Use CROSS JOIN. How many rows do you expect if there are 107 employees and 27 departments?

> **Hint:** Use: SELECT e.last_name, d.department_name FROM employees e CROSS JOIN departments d. Expected rows: 107 x 27.

Your Answer:

Part 6: SELF JOIN Tasks

A SELF JOIN is when a table is joined to itself. You use different aliases for the same table. This is very useful when a table has a relationship to itself, like employees and their managers (because managers are also employees in the same table).

Task 12: Find Each Employee and Their Manager [15 pts]

Write a query to show the **employee last name** and the **manager last name**. Use the employees table twice. Join on the manager_id column. Use aliases: **e** for employee and **m** for manager.

> **Hint:** FROM employees e JOIN employees m ON e.manager_id = m.employee_id

Your Answer:

Task 13: Find Employees Who Earn More Than Their Manager [20 pts]

Write a query to show the **employee last name**, the **manager last name**, the **employee salary**, and the **manager salary**. Show only rows where the employee earns MORE than the manager.

> **Hint:** Use a SELF JOIN and add a WHERE clause: WHERE e.salary > m.salary

Your Answer:

Part 7: Multi-Table JOIN Challenges

These tasks require joining 3 or more tables. Remember: each JOIN connects two tables. Start with one table, then add one table at a time. Think about which columns connect the tables.

Task 14: Employee Full Information (4 Tables) [20 pts]

Write a query to show: **employee last name**, **job title**, **department name**, and **city** (from locations table). Join 4 tables: employees, jobs, departments, and locations.

> **Hint:** Join: employees -> jobs (job_id), employees -> departments (department_id), departments -> locations (location_id)

Your Answer:

Task 15: Employee Location Chain (5 Tables) [20 pts]

Write a query to show: **employee last name**, **department name**, **city**, **country name**, and **region name**. You need to join 5 tables: employees, departments, locations, countries, and regions.

> **Hint:** Follow the chain: employees -> departments -> locations -> countries -> regions. Each link uses the matching ID column.

Your Answer:

Task 16: Salary Analysis with LEFT JOIN (3 Tables) [20 pts]

Write a query to show **department name**, **job title**, and **average salary** for each combination. Use LEFT JOIN so that even departments or jobs with no employees appear. Use GROUP BY and AVG(salary).

> **Hint:** Join departments -> employees -> jobs. Use LEFT JOIN. GROUP BY d.department_name, j.job_title.

Your Answer:

Task 17: Employees Who Changed Jobs (3 Tables) [20 pts]

Write a query to show the **employee last name**, their **current job title**, and their **previous job title** (from job_history). Only show employees who appear in job_history.

> **Hint:** Join employees -> jobs (current job on job_id), employees -> job_history (on employee_id), job_history -> jobs (previous job on job_id). Use two different aliases for jobs table.

Your Answer:

Part 8: Bonus Tasks

These tasks are a little harder. Try them if you finish the other tasks. They combine JOINs with other SQL features like subqueries, aggregate functions, and CASE.

Task 18: Department with Highest Average Salary [20 pts]

Write a query to find the **department name** with the **highest average salary**. Show the department name and the average salary. Join employees and departments.

> **Hint:** Use a subquery with AVG(salary) and ORDER BY ... DESC with FETCH FIRST 1 ROW ONLY or ROWNUM = 1.

Your Answer:

Task 19: Employees with Salary Above Their Job Minimum [20 pts]

Write a query to show **employee last name**, **job title**, **salary**, and **min_salary** for that job. Show only employees whose salary is **at least 20% above** the minimum salary for their job. Use JOIN with jobs table.

> **Hint:** WHERE e.salary > j.min_salary * 1.2

Your Answer:

Task 20: Complete HR Report (All 6 Tables) [25 pts]

Write a query to create a complete HR report. Show: **employee last name**, **job title**, **department name**, **city**, **country name**, and **region name**. Join all 6 HR tables: employees, jobs, departments, locations, countries, regions. Use LEFT JOIN to keep all employees even if some data is missing. Order the result by region name, then country name, then department name.

> **Hint:** This is the full chain: employees -> jobs (job_id), employees -> departments (department_id), departments -> locations (location_id), locations -> countries (country_id), countries -> regions (region_id). Use LEFT JOIN for each step.

Your Answer:

Score Summary

Part	Topic	Tasks	Total Points
1	INNER JOIN	1 - 4	50
2	LEFT JOIN	5 - 7	40
3	RIGHT JOIN	8 - 9	20
4	FULL OUTER JOIN	10	10

5	CROSS JOIN	11	10
6	SELF JOIN	12 - 13	35
7	Multi-Table Challenges	14 - 17	80
8	Bonus Tasks	18 - 20	65
Total		**20 tasks**	**310**